
Intervene on What the Agent Sees, Not on What the Model Predicts: Robust Imagination-Trained Agents under Changed Opponent Dynamics

Raban Emunds

Technical University of Darmstadt
raban.emunds@tu-darmstadt.de

Abstract

World models let agents learn entirely in imagination, but both the model and the agent inherit the correlations of the environment they were fitted to. We study this in object-centric Pong, where the opponent paddle tracks the ball. The robustness test is the `lazy_enemy` modification, in which the opponent only moves while the ball approaches it, and it is never used for training, tuning or model selection. An actor trained purely inside an object-factored world model wins unmodified Pong clearly (final score +17.1), yet it *loses* under `lazy_enemy` (-10.1). Diagnosing the failure, we find that the world model’s prediction of the ball depends on the opponent even far from any bounce. Swapping in a counterfactual opponent moves the predicted ball by up to 20 px. Sparsity penalties on the model’s interaction gates, including hard-concrete and straight-through binary gates driven by the receiving object, do not remove this dependence. As a consequence, training the actor under interventions on the opponent *inside* the world model teaches it a false causal link and brings the final score only to +0.2. We propose to intervene on the *actor’s observation* of the opponent instead: the imagined physics stays exactly as learned, while in most imagined rallies the agent sees a frozen, rescaled or random-walking opponent. Checkpoints are chosen with a selection rule that uses only the unmodified game. With this rule, actors trained this way reach a final score of $+15.3 \pm 2.1$ under `lazy_enemy` across 5 seeds (goal: $\geq +10$) and win 100% of the held-out games, while matching the baseline on the unmodified game (+17.6).

1 Introduction

Model-based reinforcement learning agents such as World Models [11], SimPLe [15], Dreamer [12, 13], IRIS [23] and DIAMOND [1] learn a model of the environment from real interaction and then train a policy on imagined rollouts. Such an agent is only as good as its model. More subtly, it is only as *robust* as its model and its policy are to changes the training data never showed. A model fitted to one environment absorbs that environment’s correlations, and an actor optimised inside the model exploits them [9, 4].

We study this problem in a setting where the change is clean, local and known in advance to the evaluator only. In JAXAtari’s object-centric Pong [14, 5], the opponent (“enemy”) paddle always moves towards the ball. The `lazy_enemy` modification [6] changes exactly one mechanism: the enemy only follows the ball while the ball is moving towards it, and stays still otherwise. Everything else, including the ball’s physics and the agent’s paddle, is unchanged. We train exclusively on the

unmodified game and ask for an agent that still wins clearly under `lazy_enemy`, with a final score (own points minus enemy points in a game to 21) of at least +10.

The failure. A strong baseline, an object-factored world model with per-step gated interactions and a PPO actor trained purely in imagination, wins the unmodified game with a final score of +17.1 (up to 21:2). Under `lazy_enemy` it loses with a final score of -10.1 (Section 5). The enemy’s position is a faithful proxy of the ball’s height in training. Both the actor and the world model learn to use it, and the proxy breaks when the enemy stops tracking the ball.

Why the obvious fixes fail. Object-centric and causal world models aim to predict each object from only the objects it interacts with [2, 16, 17, 10, 27, 26, 31]. If the world model were contained in this way, one could intervene on the enemy’s mechanism inside the model and train the actor on the resulting counterfactual games, as a form of dynamics randomisation [24, 30]. We show with a counterfactual leakage test that, in our setting, the model’s ball prediction reacts to the enemy even in free flight, and that this dependence grows sharply for enemy positions never seen in training. Hard-concrete L_0 gates [21], receiver-driven gates and straight-through binary gates [3] all fail to close it. Removing the ball’s access to the enemy entirely costs only 0.2 px of one-step accuracy. The data simply never shows that an enemy far from the ball’s height should not matter. Interventions on the enemy inside such a model therefore change the imagined ball too, and teach the actor a false causal link (final score +0.2).

Our approach. We keep the imagined dynamics exactly as learned and intervene only on what the *actor sees* of the enemy: in most imagined rallies the enemy in the actor’s input is frozen, rescaled or replaced by a random walk, while the world model, the ball and the reward are untouched (Figure 1). The actor learns to win from the ball and its own paddle while the game it experiences stays physically consistent. We select checkpoints with a proxy that uses only the unmodified game: its score when the agent’s view of the enemy is frozen.

Contributions.

- A clean protocol for studying robustness of imagination-trained agents to a changed object mechanism, with a strict separation of training/selection (unmodified Pong) and test (`lazy_enemy`).
- A counterfactual *leakage* test for object-factored world models, and evidence that sparsity-regularised gating does not make the model invariant to out-of-distribution behaviour of a correlated object.
- *Observation interventions in imagination*: a simple change to imagination training that makes the actor robust ($+15.3 \pm 2.1$ under `lazy_enemy`, 5 seeds, 100% of games won) without requiring a causally correct world model.
- A selection proxy on the unmodified game (frozen enemy view) that flags agents relying on the enemy without touching the test modification. It is a necessary check, but, as we show, not a sufficient one.

2 Related work

World models and imagination training. Learning behaviour inside learned models [11] has become competitive on Atari with SimPLe [15], DreamerV3 [13], IRIS [23] and DIAMOND [1]. These works model pixels or latent states of the whole scene and evaluate in the training environment. We use object-centric states and ask what happens when one object’s mechanism changes at test time.

Object-centric and structured dynamics. Interaction networks [2], NRI [16], C-SWM [17], SlotFormer [32] and object-centric RL [33, 7] factor the state into objects and their interactions. Recurrent independent mechanisms [10] and the sparse-mechanism-shift view of causal representation learning [27] argue that such factorisations should make models robust to local changes. Locally factored dynamics have been exploited for counterfactual data augmentation (CoDA, [26]) and causal dynamics learning [31]. Our leakage analysis shows a practical limit: when two objects are strongly correlated in the training data, sparsity alone does not identify the causal parents.

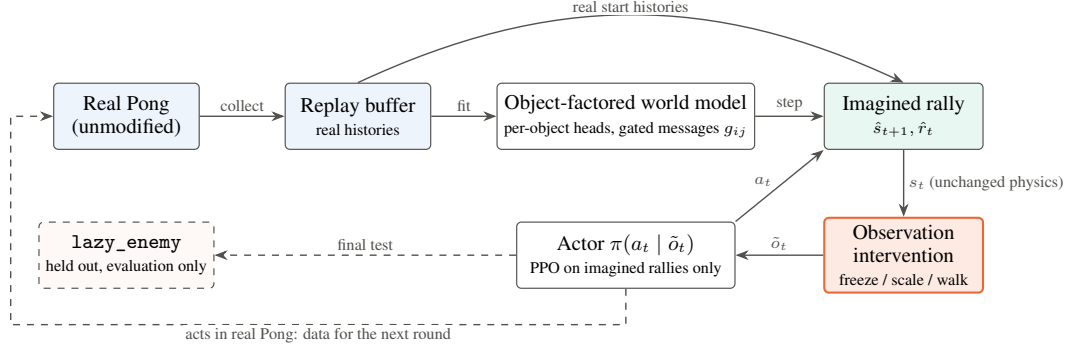


Figure 1: **Overview.** Real interaction with unmodified Pong only fills a replay buffer. An object-factored world model is fitted on it, and the actor is trained with PPO purely on imagined rallies that start from real histories. Our change (orange) is applied between the imagined state and the actor: in most rallies the enemy’s position *as seen by the actor* is intervened on, while the imagined dynamics, ball and reward are not. `lazy_enemy` is used only to evaluate the finished actor.

Robustness and shortcuts in RL. Causal confusion [4] and shortcut learning [9] describe policies that rely on spurious but predictive features. Domain and dynamics randomisation [30, 24], adversarial dynamics [25], observation augmentation [19, 20] and disentangled representations [8] are common remedies. They are surveyed in [18]. HackAtari [6] introduced game modifications, among them `lazy_enemy`, to expose such shortcuts in Atari agents. Our method is a form of observation randomisation, but it is applied *inside imagination* and targeted at one object. We show that applying the same perturbations to the imagined dynamics instead is harmful when the world model is not contained.

3 Problem setting

Environment. We use JAXAtari’s Pong [14] with its standard object-centric pipeline: sticky actions ($p = 0.25$), up to 30 no-op resets, frame skip 4 and a stack of the last 4 object-centric frames [22]. From each frame we use the (x, y) position of the three objects, player, enemy and ball, so $s_t \in \mathbb{R}^{3 \times 2}$. There are 6 actions. A game ends when one side reaches 21 points, and its *final score* is the agent’s points minus the enemy’s.

The changed mechanism. In Pong, the enemy moves 2 px per frame towards the ball’s height, pausing every eighth frame: $y_{\tau+1}^e = y_{\tau}^e + 2 \text{sign}(y_{\tau}^b - y_{\tau}^e) \cdot \mathbb{I}[\tau \bmod 8 \neq 0]$. `lazy_enemy` multiplies the update by $\mathbb{I}[v_{x,\tau}^b < 0]$: the enemy only moves while the ball travels towards it, and stays wherever it last hit the ball otherwise. All other mechanisms are unchanged. A policy that relies on the ball and its own paddle is not affected by this change, and the change makes the enemy weaker. An agent that fails under `lazy_enemy` does so because it relies on the enemy’s normal behaviour.

Protocol. All real data comes from unmodified Pong. The actor is trained *only* in imagination. Hyperparameters, stopping and checkpoint selection use only unmodified Pong. Each selected checkpoint is evaluated once on 32 full games of Pong and of `lazy_enemy` with the same 32 seeds. *Success* is an average final score $\geq +10$ under `lazy_enemy`.

4 Method

4.1 Object-factored world model

The world model f_{θ} maps a history of $K = 16$ object states and the action to the next object states and the reward. Each object $i \in \{\text{player, enemy, ball}\}$ has its own encoder $e_i = \text{MLP}_i(\text{positions and velocities of } i \text{ over } K \text{ frames})$, and the action is embedded as a fourth token e_a . Object i receives a message from every other token j , scaled by a gate that is recomputed at

every step:

$$m_i = \sum_{j \neq i} g_{ij} \text{LN}(\text{MLP}_i^{\text{msg}}([e_i, e_j])), \quad g_{ij} = \sigma(\text{MLP}_i^{\text{gate}}([e_i, e_j])) \in [0, 1]. \quad (1)$$

A per-object head predicts the displacement $\Delta \hat{s}_i = \text{MLP}_i^{\text{dec}}([e_i, m_i])$, and the ball’s head also predicts the reward $\hat{r} \in \{-1, 0, +1\}$, since points are events of the ball. The gates are the model’s dependency graph and are regularised with an L_1 penalty. We train with an 8-step open-loop loss in which predictions are fed back as inputs: $\mathcal{L} = \sum_{k=1}^8 [\sum_i \|\Delta \hat{s}_{i,t+k} - \Delta s_{i,t+k}\|^2 + \text{CE}(\hat{r}_{t+k}, r_{t+k}) + \lambda \sum_{i \neq j} g_{ij,t+k}]$. The model learns sensible, time-varying dependencies (Figure 9): the player listens to the action, and the ball listens to the player and the action only around the player’s hits.

4.2 Actor training in imagination

We treat the world model as a batched JAX environment [14]. An imagined episode starts from a real 16-frame history sampled from the replay buffer and lasts one *rally*: it ends when the reward head predicts a point, or after 256 steps with value bootstrapping. Ball and enemy positions are rounded to integers, as in the game. The actor-critic (two 2×256 MLPs) sees the last four frames of object positions and their differences, and is trained with PPO [29] and GAE [28] on 2048 imagined environments. Training proceeds in 5 Dreamer-style rounds [12]. Each round collects 512k real steps of *unmodified* Pong (round 0 with a random policy, later rounds with the current actor and 10% random actions), refits the world model on all data, and runs 400 PPO updates (105M imagined steps) in imagination. The actor never sees a real transition.

4.3 Observation interventions

The actor’s input is computed from the imagined state s_t with one change: the enemy’s height is replaced by an *observed* enemy track $\tilde{y}_t^e$. At the start of each imagined rally we sample a mode c : *none* with probability p_{none} (then $\tilde{y}^e = y^e$), otherwise one of three generic interventions on the observed enemy’s motion. Writing $\Delta_t = y_{t+1}^e - y_t^e$ for the imagined enemy’s displacement,

$$\tilde{y}_{t+1}^e = \text{clip}(\tilde{y}_t^e + \tilde{\Delta}_t), \quad \tilde{\Delta}_t = \begin{cases} (1 - z_t) \Delta_t & \text{freeze; } z_t \in \{0, 1\} \text{ flips w.p. } 0.1, \\ \text{round}(\kappa \Delta_t) & \text{scale; } \kappa \sim \mathcal{U}[0, 1.5], \\ \delta_t \sim \mathcal{U}\{-8, -4, 0, 4, 8\} & \text{random walk.} \end{cases} \quad (2)$$

The switch z_t flips independently at every step, κ is drawn once per rally, and the non-*none* modes are drawn in ratio 2:1:1. Crucially, the world model, the ball, the reward and the *true* enemy are never changed, so every imagined rally is a consistent game of Pong under the learned dynamics. The actor experiences games in which its view of the enemy is unreliable, and is rewarded for winning them. We use $p_{\text{none}} = 0.1$.

Design choice. Intervening on the enemy object encodes the assumption that the opponent is the part of the environment whose behaviour may change. We intervene on the object that the agent neither controls nor has to hit, and not on the ball, since without the ball the game cannot be played. The interventions are generic perturbations of one object’s motion. They are not the `lazy_enemy` rule, which conditions the enemy’s motion on the ball’s direction.

4.4 Selection without the test modification

Each run yields 10 candidates (5 rounds $\times$ greedy/sampled actions). We choose the one with the highest mean of two final scores, both measured on unmodified Pong (32 games each): normal play, and play in which the agent’s view of the enemy is frozen at a random height for the whole game, a generic sensor-failure check. The rule was fixed before the runs reported for our main configuration, and we apply it to every method.

5 Experiments

Methods compared. **WM + MLP actor** (Exp. 1): the world model of Section 4.1 and PPO in imagination, without interventions. **Dynamics interventions** (Exp. 2): the same pipeline with L_0

Algorithm 1 Imagination training with observation interventions (one run)

```
1:  $\mathcal{D} \leftarrow \emptyset$ ; initialise world model  $f_\theta$ , actor-critic  $(\pi_\phi, V_\phi)$ 
2: for round = 0, ..., 4 do
3:   collect 512k steps of unmodified Pong with a random policy (round 0) or  $\pi_\phi$  (+10% random), add to  $\mathcal{D}$ 
4:   fit  $f_\theta$  on  $\mathcal{D}$  with the 8-step open-loop loss (Section 4.1)
5:   for 400 PPO updates do
6:     for each of 2048 imagined envs, 128 steps do
7:       if new rally then sample a real 16-frame history from  $\mathcal{D}$  and a mode  $c$  (Eq. 2);  $\tilde{y}^e \leftarrow y^e$ 
8:       end if
9:        $\tilde{o}_t \leftarrow$  actor input from  $s_{t-3:t}$  with the enemy's height replaced by  $\tilde{y}_{t-3:t}^e$ 
10:       $a_t \sim \pi_\phi(\cdot | \tilde{o}_t)$ ;  $(\hat{s}_{t+1}, \hat{r}_t) \leftarrow f_\theta(s_{t-15:t}, a_t)$   $\triangleright$  dynamics never intervened
11:      update  $\tilde{y}_{t+1}^e$  from the imagined enemy's displacement and mode  $c$  (Eq. 2)
12:      end the rally if  $\hat{r}_t \neq 0$  or after 256 steps (bootstrap with  $V_\phi$ )
13:    end for
14:    PPO update of  $(\pi_\phi, V_\phi)$  on the imagined batch
15:  end for
16:  save candidate checkpoints (greedy and sampled actions)
17: end for
18: select a candidate with the unmodified-Pong rule (Section 4.4); evaluate once on lazy_enemy
```

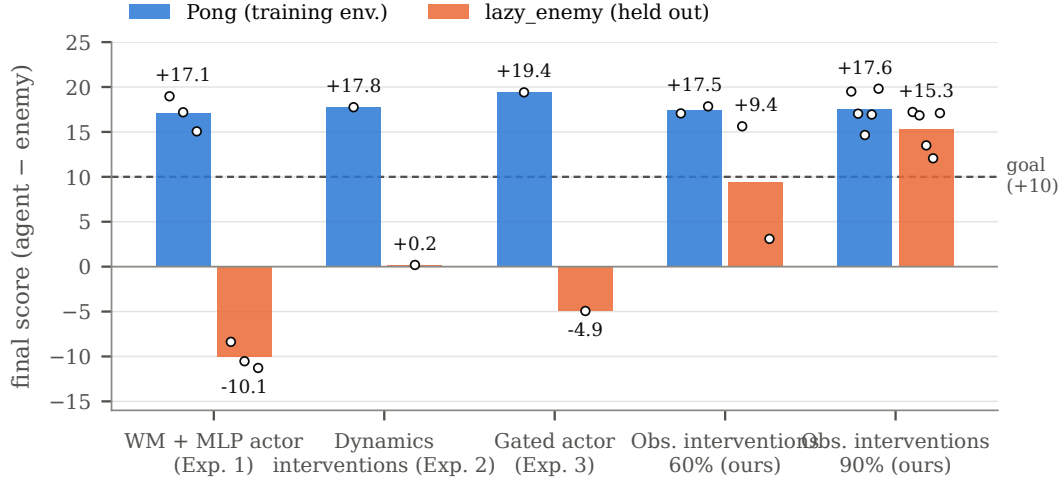


Figure 2: **Final score on unmodified Pong and under the held-out lazy_enemy modification.** Bars: mean over seeds; dots: individual seeds (32 games each); dashed: success threshold +10. All methods win unmodified Pong clearly. Only observation interventions stay robust under lazy_enemy.

gates [21], where the interventions of Eq. (2) are applied to the *world model's* enemy prediction (with probability 0.75 per rally), so the imagined ball reacts to the intervened enemy. **Gated actor** (Exp. 3): an actor whose ball and enemy inputs pass through per-step hard-concrete gates with an expected- L_0 penalty, so it can learn to ignore the enemy. **Observation interventions** (ours) with $p_{\text{none}} = 0.4$ and 0.1. All methods share the data pipeline, PPO settings and selection rule, and all except Exp. 2 share the world model. Each run takes about 40 minutes on one A100 GPU. We report the mean and standard deviation over seeds, and all 32 games per seed.

5.1 Main result

Figure 2 and Table 1 summarise the results. Imagination training works: every method wins unmodified Pong clearly, although its actor never experienced a real transition. The baseline's strength is brittle. Under lazy_enemy it loses (-10.1 ± 1.2) , and the gated actor fares no better (-4.9) : its enemy gate stays open, because in unmodified Pong the enemy is genuinely informative. Dynamics interventions improve the lazy_enemy score to +0.2 but fall far short of the goal. Observation interventions with $p_{\text{none}} = 0.1$ reach $+15.3 \pm 2.1$ (range +12.1 to +17.2 over seeds; worst single game

Table 1: **Main results.** Final score (agent – enemy points, mean $\pm$ std over seeds, 32 games per seed). “Frozen view” is the selection proxy on unmodified Pong. *lazy_enemy* is the held-out test.

Method	seeds	Pong	Pong, frozen view	<i>lazy_enemy</i>	won
WM + MLP actor (Exp. 1)	3	+17.1 $\pm$ 1.6	−13.4 $\pm$ 2.0	−10.1 $\pm$ 1.2	0%
Dynamics interventions (Exp. 2)	1	+17.8	+11.3	+0.2	53%
Gated actor (Exp. 3)	1	+19.4	−1.3	−4.9	6%
Obs. interventions 60% (ours)	2	+17.5 $\pm$ 0.4	+17.9 $\pm$ 0.6	+9.4 $\pm$ 6.3	89%
Obs. interventions 90% (ours)	5	+17.6 $\pm$ 1.9	+18.0 $\pm$ 1.7	+15.3 $\pm$ 2.1	100%

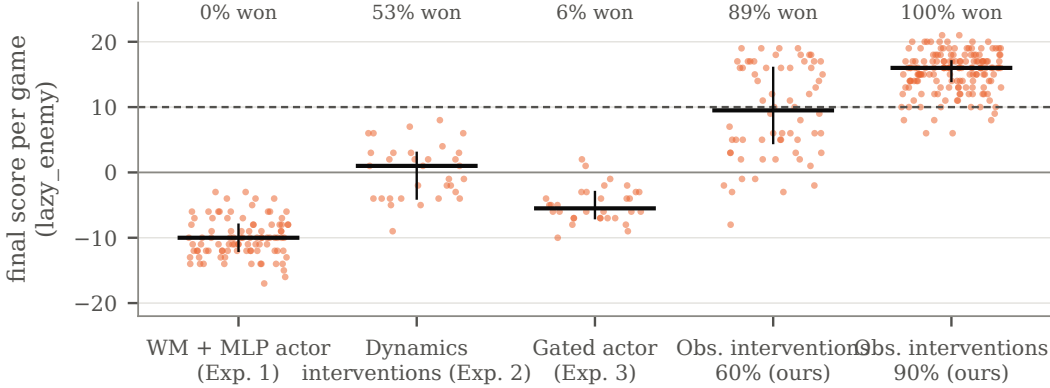


Figure 3: **Final score of every *lazy_enemy* game** (all seeds pooled; bar: median, whisker: interquartile range; top: share of games won).

+6) and win 100% of all *lazy_enemy* games, while scoring +17.6 on unmodified Pong. Figure 3 shows the distribution over individual games: the baselines lose most *lazy_enemy* games, while our agents win nearly all of them by large margins. Figure 4 illustrates the failure. In normal Pong, the enemy’s height tracks the ball’s while the ball travels towards the player, and the baseline’s paddle is drawn towards the enemy’s height even when it no longer says anything about the ball.

5.2 Why intervening inside the world model fails: leakage

If the world model were object-contained, interventions on its enemy (Exp. 2) would produce valid counterfactual games. We test containment with a *counterfactual leakage* test: on real held-out windows we replace object j ’s entire history by that of a random other window, and measure how much object i ’s one-step prediction changes. For the ball and the enemy, a contained model should change only near the enemy’s paddle. Figure 5 shows three findings. (a) Removing the ball’s messages from the enemy entirely costs little: the mean one-step error rises from 0.71 to 0.90 px, mostly in the bounce zone. (b) Yet the model uses the enemy when the ball is far away, and the effect grows with the enemy-ball gap after the swap. Gaps above 40 px never occur in real data, where the enemy tracks the ball, and there the ball’s prediction moves by more than 20 px. (c) Pairwise L_0 gates, receiver-driven L_0 gates, receiver-driven soft gates with exact zeros, and receiver-driven straight-through binary gates all keep a far-field leakage of 10–14 px. Receiver-driven L_0 gates even collapsed to state-independent constants and stopped the ball from listening to the player. A penalty on the training loss cannot tell “useful in training” from “causally needed” when two objects are this correlated. Consequently, intervening on the enemy inside the model also moves the imagined ball, and the actor learns that the enemy matters for the ball’s trajectory. Observation interventions avoid the problem: they never feed an unseen enemy into the world model.

5.3 The world model is not robust, but the actor is

Figure 8 shows the open-loop error of our agent’s world model on real trajectories. Under *lazy_enemy*, the enemy’s prediction is wrong, as expected for a changed mechanism. Because of the leakage above, the ball’s prediction degrades too (1.0 px to 4.2 px after one step). The world model is therefore *not* robust. The actor is robust anyway, because it was trained to succeed under

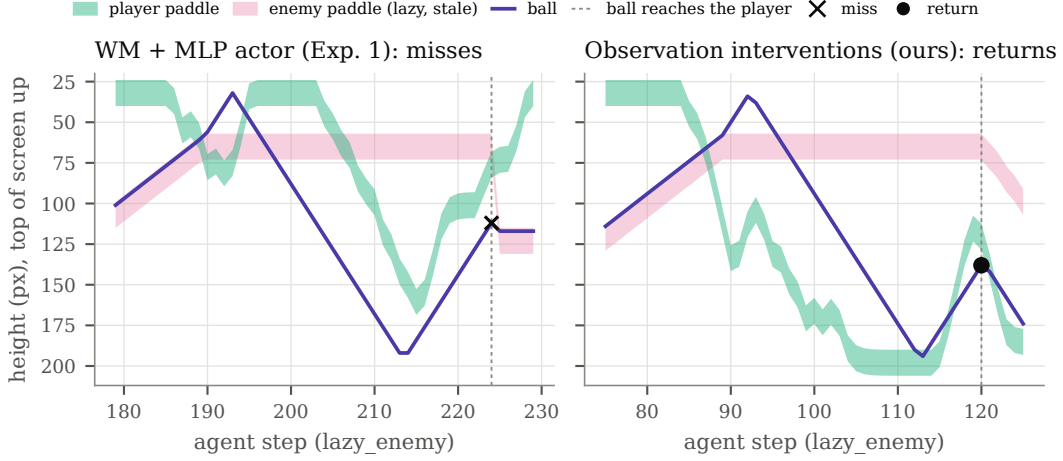


Figure 4: **Behaviour under lazy_enemy.** Left: the baseline’s first lost point in a lazy_enemy game. After returning the ball, the lazy_enemy enemy (magenta) stays where it hit it. The baseline’s paddle (aqua) first follows the ball down, then moves back up towards the stale enemy’s height and misses the ball by a few pixels. All three points it lost in the first 600 steps of this game look alike. Right: our agent in a comparable situation (enemy ≥ 40 px from the ball when it arrives) stays with the ball and returns it.

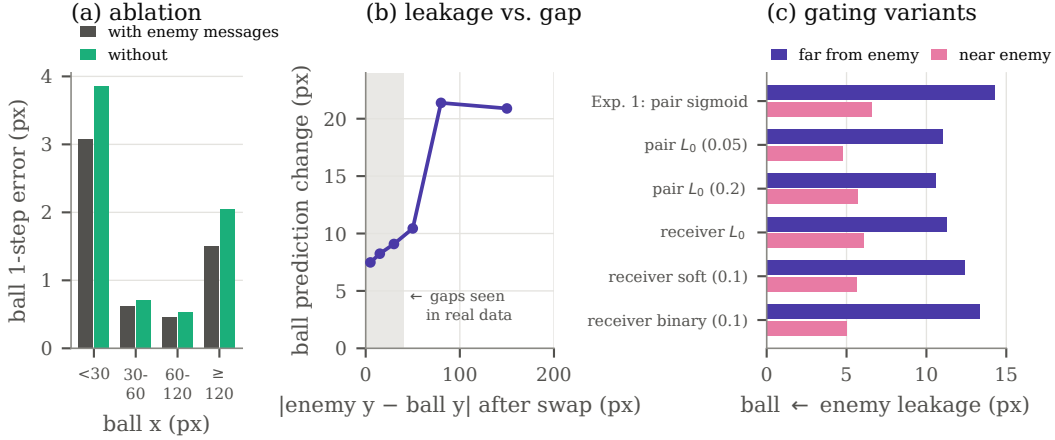


Figure 5: **World-model leakage from enemy to ball.** (a) Ablation: the ball’s one-step error with and without messages from the enemy, by ball position. (b) Change of the ball’s prediction when the enemy’s history is swapped, by the resulting enemy-ball gap (ball moving, far from the enemy). The shaded band covers the gaps that occur in real data. (c) Far-field and near-field leakage for gating variants.

unreliable enemy information while the imagined ball stayed consistent. The gates of the world model (Figure 9) remain interpretable. The ball listens to the player and the action only around the player’s hits, and the enemy listens to the ball. The ball also listens to the enemy whenever the ball approaches it, which is the far-field dependence measured above.

5.4 Ablations and analysis

Fraction of intervened rallies. With $p_{\text{none}} = 0.4$ (interventions in 60% of rallies) the result depends on the seed. One seed is robust and the other still loses many lazy_enemy games ($+9.4 \pm 6.3$; Table 1). The agent can learn to trust the enemy when its track “looks normal”. With interventions in 90% of rallies all seeds are robust. More interventions did not cost performance on the unmodified game (Figure 6).

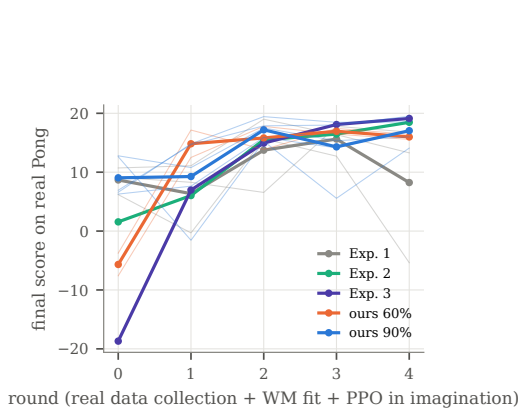


Figure 6: **Real Pong final score after each round** (greedy actor; thin: seeds, thick: mean).

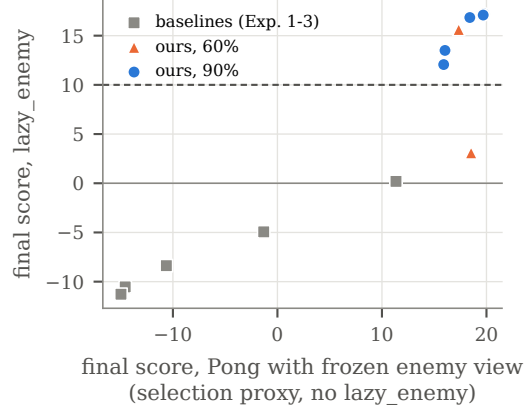


Figure 7: **Selection proxy vs. held-out score.** Each point is one selected run.

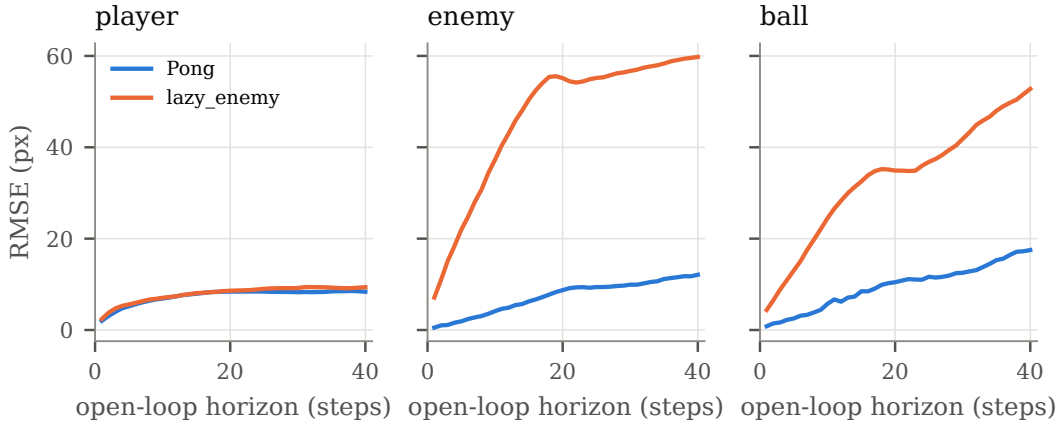


Figure 8: **Open-loop world-model error** (our agent’s world model, 4096 windows of its own real trajectories) on unmodified Pong and under `lazy_enemy`.

The selection proxy. Figure 7 plots the frozen-view score, measured on the unmodified game, against the held-out `lazy_enemy` score for every selected run. Agents that collapse when their view of the enemy is frozen also fail under `lazy_enemy`. The clearest case is the baseline (seed 0): +19.0 on Pong, −10.6 with a frozen enemy view, and −8.4 under `lazy_enemy`. The proxy is therefore a useful, test-free warning sign. It is not sufficient, however. One run with 60% interventions is unaffected by the frozen view (+18.5) but reaches only +3.1 under `lazy_enemy`: an enemy that is frozen for the whole game is a different perturbation from one that is sometimes still and sometimes moving. In all our runs the proxy selected the same checkpoint as the plain Pong score, so it did not change any reported result.

Imagined vs. real robustness. In imagination, our actor wins the same share of points with and without an intervened enemy view (e.g. 0.94 vs. 0.94 in the last round of seed 0), so its imagined success does not depend on the enemy. Appendix B shows imagined rollouts next to reality.

6 Discussion and limitations

What the result does and does not show. Our agents are robust to a changed opponent mechanism because they were trained not to rely on the opponent’s position, and imagination made that training cheap and physically consistent. The world model itself is not object-contained, and our leakage analysis suggests that sparsity regularisation alone will not make it so when objects are

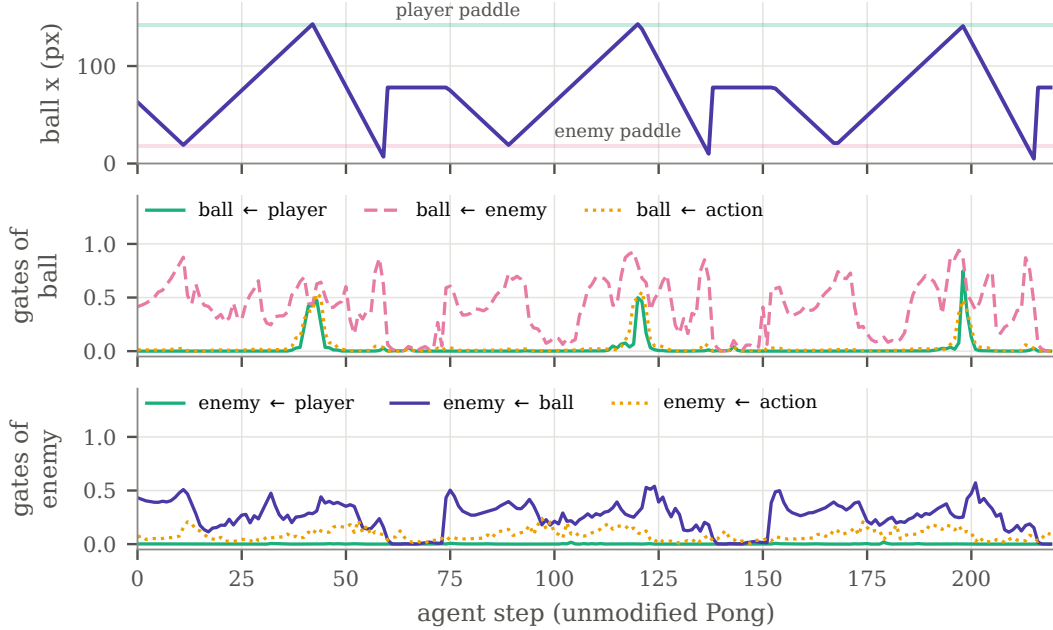


Figure 9: **Interaction gates of the world model along a real Pong trajectory.** Top: ball x with the paddles’ x ranges. Middle: gates of the ball. Bottom: gates of the enemy.

strongly correlated in the training data. Interventional data, or stronger priors such as locality, would be needed.

Assumptions and possible leakage of test knowledge. Choosing the enemy as the object to intervene on encodes a prior about which part of the environment may change. Our interventions are generic (freeze, scaled motion, random walk), but the *freeze* family resembles the stale enemy of *lazy_enemy* more than, e.g., an enemy that moves faster than the ball. The fraction $p_{\text{none}} = 0.1$ was chosen after observing the $p_{\text{none}} = 0.4$ runs on *lazy_enemy*, so the main configuration is not a fully blind test of that hyperparameter. For this reason we report both settings. The selection rule was fixed before the main runs, and every selected checkpoint was evaluated on *lazy_enemy* once.

Scope. We study one game, one modification and a small number of seeds, with ground-truth object states from JAXAtari rather than learned object representations. Whether observation interventions scale to games with many interacting objects, where the object to intervene on is less obvious, remains open. A natural extension is to intervene on every non-controlled object in turn and to use the frozen-view proxy per object to find the objects the agent should not rely on.

7 Conclusion

An imagination-trained agent is only as robust as the correlations it was allowed to exploit. In object-centric Pong, the opponent’s position is such a correlation, and it is absorbed both by the world model and by the actor. Making the world model object-contained through sparsity did not remove it. Intervening on the model’s dynamics was harmful, because the leaking model turns those interventions into false physics. Intervening on what the actor *sees*, while keeping the imagined physics intact, produced agents that win the held-out *lazy_enemy* game with a final score of $+15.3 \pm 2.1$ across 5 seeds, without ever training, tuning or selecting on it.

Acknowledgments and Disclosure of Funding

Experiments, code and a first draft of this paper were produced with the help of an AI coding assistant (Claude, Anthropic) working under the author’s direction. All numbers are generated from the released evaluation files.

References

- [1] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, and F. Fleuret. Diffusion for world modeling: Visual details matter in Atari. In *Advances in Neural Information Processing Systems*, 2024.
- [2] P. Battaglia, R. Pascanu, M. Lai, D. J. Rezende, and K. Kavukcuoglu. Interaction networks for learning about objects, relations and physics. In *Advances in Neural Information Processing Systems*, 2016.
- [3] Y. Bengio, N. Léonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [4] P. de Haan, D. Jayaraman, and S. Levine. Causal confusion in imitation learning. In *Advances in Neural Information Processing Systems*, 2019.
- [5] Q. Delfosse, J. Blüml, B. Gregori, S. Sztwiertnia, and K. Kersting. OCArari: Object-centric Atari 2600 reinforcement learning environments. *arXiv preprint arXiv:2306.08649*, 2023.
- [6] Q. Delfosse, J. Blüml, B. Gregori, and K. Kersting. HackAtari: Atari learning environments for robust and continual reinforcement learning. *arXiv preprint arXiv:2406.03997*, 2024.
- [7] Q. Delfosse, S. Sztwiertnia, M. Rothermel, W. Stammer, and K. Kersting. Interpretable concept bottlenecks to align reinforcement learning agents. In *Advances in Neural Information Processing Systems*, 2024.
- [8] M. Dunion, T. McInroe, K. S. Luck, J. P. Hanna, and S. V. Albrecht. Conditional mutual information for disentangled representations in reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- [9] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.
- [10] A. Goyal, A. Lamb, J. Hoffmann, S. Sodhani, S. Levine, Y. Bengio, and B. Schölkopf. Recurrent independent mechanisms. In *International Conference on Learning Representations*, 2021.
- [11] D. Ha and J. Schmidhuber. Recurrent world models facilitate policy evolution. *Advances in Neural Information Processing Systems*, 2018.
- [12] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- [13] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [14] JAXAtari contributors. JAXAtari: JAX-based object-centric Atari environments. <https://github.com/k4ntz/JAXAtari>, 2025.
- [15] Ł. Kaiser, M. Babaeizadeh, P. Miłos, B. Osiński, R. H. Campbell, K. Czechowski, D. Erhan, C. Finn, P. Kozakowski, S. Levine, et al. Model-based reinforcement learning for Atari. In *International Conference on Learning Representations*, 2020.
- [16] T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. Zemel. Neural relational inference for interacting systems. In *International Conference on Machine Learning*, 2018.
- [17] T. Kipf, E. van der Pol, and M. Welling. Contrastive learning of structured world models. In *International Conference on Learning Representations*, 2020.
- [18] R. Kirk, A. Zhang, E. Grefenstette, and T. Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.
- [19] M. Laskin, K. Lee, A. Stooke, L. Pinto, P. Abbeel, and A. Srinivas. Reinforcement learning with augmented data. In *Advances in Neural Information Processing Systems*, 2020.

- [20] K. Lee, K. Lee, J. Shin, and H. Lee. Network randomization: A simple technique for generalization in deep reinforcement learning. In *International Conference on Learning Representations*, 2020.
- [21] C. Louizos, M. Welling, and D. P. Kingma. Learning sparse neural networks through L_0 regularization. In *International Conference on Learning Representations*, 2018.
- [22] M. C. Machado, M. G. Bellemare, E. Talvitie, J. Veness, M. Hausknecht, and M. Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.
- [23] V. Micheli, E. Alonso, and F. Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023.
- [24] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation*, 2018.
- [25] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta. Robust adversarial reinforcement learning. In *International Conference on Machine Learning*, 2017.
- [26] S. Pitis, E. Creager, and A. Garg. Counterfactual data augmentation using locally factored dynamics. In *Advances in Neural Information Processing Systems*, 2020.
- [27] B. Schölkopf, F. Locatello, S. Bauer, N. R. Ke, N. Kalchbrenner, A. Goyal, and Y. Bengio. Toward causal representation learning. *Proceedings of the IEEE*, 109(5):612–634, 2021.
- [28] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.
- [29] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [30] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017.
- [31] Z. Wang, X. Xiao, Z. Xu, Y. Zhu, and P. Stone. Causal dynamics learning for task-independent state abstraction. In *International Conference on Machine Learning*, 2022.
- [32] Z. Wu, N. Dvornik, K. Greff, T. Kipf, and A. Garg. SlotFormer: Unsupervised visual dynamics simulation with object-centric models. In *International Conference on Learning Representations*, 2023.
- [33] A. Zadaianchuk, M. Seitzer, and G. Martius. Self-supervised visual reinforcement learning with object-centric representations. In *International Conference on Learning Representations*, 2021.

A Hyperparameters

Table 2: Hyperparameters (shared by all methods unless noted).

<i>Environment</i>	
JAXAtari Pong, object-centric wrapper	frame skip 4, frame stack 4, sticky actions 0.25, ≤ 30 no-op resets
State used	(x, y) of player, enemy, ball per frame
<i>Data collection (per round)</i>	
Rounds	5
Real steps per round	256 envs $\times$ 2000 steps = 512k (round 0: random policy)
Collection policy (rounds ≥ 1)	current actor, 10% random actions
<i>World model</i>	
History K / open-loop training horizon H	16 / 8
Width	128 (encoders, messages, decoders: 2-layer MLPs, GELU)
Gate penalty λ	0.01 (L_1 on sigmoid gates)
Optimiser	AdamW, lr $3 \cdot 10^{-4}$, weight decay 10^{-4} , grad-norm clip 1.0
Batch / steps per round	512 windows / 20k (continues from the previous round)
<i>Imagination and PPO</i>	
Imagined envs / rollout length	2048 / 128 steps
Episode	one rally (ends at a predicted point) or 256 steps (value bootstrap)
PPO updates per round	400 (≈ 105 M imagined steps)
Epochs / minibatches / clip	4 / 8 / 0.2
lr / γ / GAE λ	$3 \cdot 10^{-4}$ / 0.99 / 0.95
Entropy / value coefficients	0.01 / 0.5
Actor, critic	separate 2×256 ReLU MLPs; input: 4 frames of positions + differences (42 dims)
<i>Observation interventions (ours)</i>	
p_{none}	0.1 (ablation: 0.4)
Mode ratio freeze : scale : random walk	2 : 1 : 1
Freeze switch prob. / scale range / walk steps	0.1 per step / $\mathcal{U}[0, 1.5]$ / $\{-8, -4, 0, 4, 8\}$ px
<i>Selection and evaluation</i>	
Candidates	5 rounds $\times$ {greedy, sampled}
Selection score	mean of Pong and frozen-view Pong final score (32 games each)
Evaluation	32 full games per environment, max 10,000 agent steps

B Imagination vs. reality

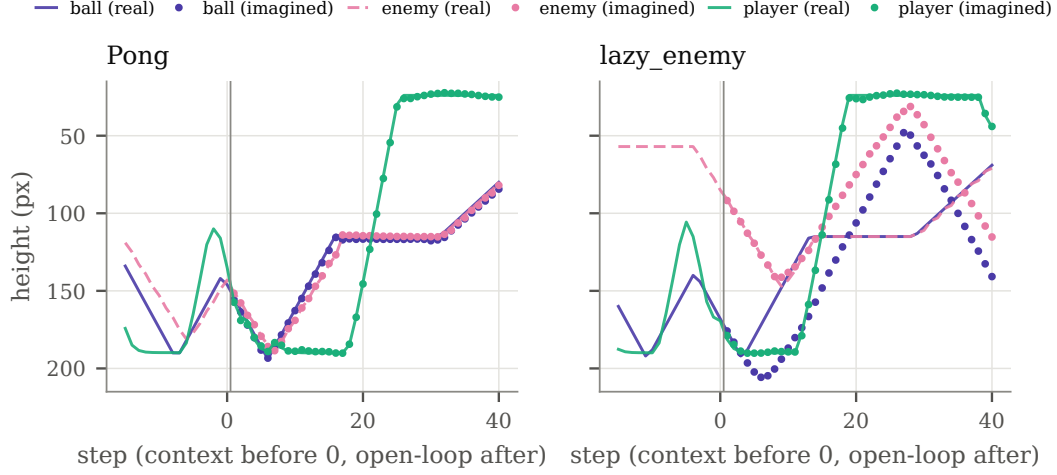


Figure 10: **Open-loop imagination** of our agent’s world model (dots) against the real trajectory (lines), 16 frames of real context followed by 40 imagined steps with the real actions. On unmodified Pong the imagined ball and paddles stay close to reality for the whole window. Under `lazy_enemy` the imagined enemy keeps tracking the ball as it did in training while the real enemy stays still, and the imagined ball diverges from the real one after roughly ten steps. The world model does not transfer to the changed game.

C Further negative results

Gated actor. Penalising the actor’s attention to objects (hard-concrete gates with an expected- L_0 penalty) either closed *all* gates before the actor had learned to play, when the penalty was active from the start (the actor lost 0.3:21), or none of them, when the penalty was applied after a warm-up. A reward-driven penalty cannot remove information that is useful in training.

Receiver-driven gates. Deciding the gate g_{ij} from the receiver’s state only, $g_{ij} = \sigma(f_i(e_i))_j$, guarantees that a closed gate makes i invariant to j . With hard-concrete gates, the gates collapsed to constants. With soft gates (zeroed below 0.1 at use time) or straight-through binary gates, they stayed open for $\text{ball} \leftarrow \text{enemy}$ in free flight (Figure 5c). The ball’s error was similar in all variants, so the open gate buys very little accuracy. It nonetheless survives the penalty, which suggests that the optimisation finds the enemy a convenient, low-noise source of the ball’s height.

Compute. All experiments ran on two A100 GPUs. One training run takes about 40 minutes, and selection plus evaluation take about 30 minutes per run.

NeurIPS Paper Checklist

1. **Claims.** Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope? [\[Yes\]](#) The claims are restricted to object-centric Pong and the lazy_enemy modification (Sections 3, 5, 6).
2. **Limitations.** [\[Yes\]](#) Section 6.
3. **Theory assumptions and proofs.** [\[NA\]](#) No theoretical results.
4. **Experimental result reproducibility.** [\[Yes\]](#) Section 4, Appendix A; the code, commands and evaluation files are released with the paper.
5. **Open access to data and code.** [\[Yes\]](#) All data is generated by the released code from the open-source JAXAtari environment.
6. **Experimental setting/details.** [\[Yes\]](#) Appendix A.
7. **Experiment statistical significance.** [\[Yes\]](#) Mean and standard deviation over seeds, per-seed points and per-game distributions (Figures 2, 3). The number of seeds is small, and some base-lines have a single seed.
8. **Experiments compute resources.** [\[Yes\]](#) Appendix C (Compute).
9. **Code of ethics.** [\[Yes\]](#)
10. **Broader impacts.** [\[NA\]](#) Foundational research on robustness in a video game, with no direct societal impact beyond general progress in RL robustness.
11. **Safeguards.** [\[NA\]](#)
12. **Licenses for existing assets.** [\[Yes\]](#) JAXAtari [14] is cited; its license applies to the environment.
13. **New assets.** [\[NA\]](#)
14. **Crowdsourcing and research with human subjects.** [\[NA\]](#)
15. **IRB approvals.** [\[NA\]](#)
16. **Declaration of LLM usage.** [\[Yes\]](#) An AI coding assistant was used to implement the experiments and draft the paper (Acknowledgements).